



Høgskulen
på Vestlandet

Safety first



DAT108 - Programmering og webapplikasjoner

F09 – Trådsikkerhet, Callable, Future

Lars-Petter Helland
Bergen
14. september 2026

Oppsummert fra modulen så langt

› F06

- › Hva er multithreading? Hvorfor? Thread og Runnable.
- › Starte, pause, stoppe og vente på tråder i Java
- › Delte ressurser, kritisk seksjon, race condition, synchronized
- › Trådsikker delt ressurs med synchronized

› F07

- › Trådsignaler. wait(), notifyAll(). Tråders livssyklus.
- › Blokkerende delt ressurs med wait() før return

› F08

- › Begreper: Dead lock, live lock, starvation, slipped condition
- › Låser. Reentrancy.
- › Lock, ReentrantLock, Condition, lock(), unlock(), await(), signalAll()



Plan for i dag

- › Trådsikkerhet, det litt større bildet
 - › Fra Java Concurrency and Multithreading Tutorial (<http://tutorials.jenkov.com/java-concurrency/index.html>):
 - › Java Memory Model
 - › Java Volatile Keyword
 - › Blocking Queues + Java » Java Concurrency Utils » Java BlockingQueue
 - › Java » Java Concurrency Utils » AtomicInteger
 - › What is Thread-Safety and How to Achieve it? (av Alejandro Ugarte)
<https://www.baeldung.com/java-thread-safety>
-
- › Hva hvis vi trenger at en tråd returnerer en verdi?
 - › Java Callable and Future Tutorial (av Rajeev Singh)
<https://www.callicoder.com/java-callable-and-future-tutorial/>



Trådsikkerhet

- › **Trådsikkerhet** betyr at kode fungerer korrekt når den blir kjørt av flere tråder samtidig.
- › Når kode er trådsikker, garanterer den at flere tråder kan **lese og skrive til de samme dataene** uten at det oppstår datakorupsjon, uventet oppførsel eller systemkrasj.
- › Vi har hittil sett på **bruk av låser** (synchronized / lock) for å hindre at tråder endrer data slik at resultatet blir feil.
- › Generelt kan vi si at **trådsikkerhet** (i Java sin minnemodell) handler om:
 - › **Atomisitet** (atomicity) – At en eller flere operasjoner utføres i ett udelelig steg, eksempel `teller++`. Det er dette vi har prøvd å oppnå med `synchronized`. I dag skal vi også se på et alternativ til låser, det vi kaller atomiske typer.
 - › **Synlighet** (visibility) - At når én tråd gjør en endring i minnet, er dette synlig for andre tråder med en gang. Pga. hurtigbuffer i datamaskinen, er ikke dette alltid tilfelle. Vi skal se et eksempel i dag der synlighet kan oppnås ved å bruke kodeordet `volatile`.
 - › **Instruksjonsrekkefølge** (ordering) – At operasjoner i koden faktisk blir utført i den rekkefølgen du skrev dem. Skal ikke gå veldig detaljert inn på dette.



Stoppe en tråd som går i en løkke – `volatile`

IntelliJ-prosjekt: 📁 `f09-eksempler`

Pakke for eksempelet: 📦 `eks1_volatileOgSynlighet`



TelleOpp og TelleNed – AtomicInteger

IntelliJ-prosjekt: 📁 f09-eksempler

Pakke for eksempelet: 📦 eks2_telleOppTelleNed



Stoppe en tråd som «sover» - `interrupt()`

- › Siden vi så hvordan stoppe en tråd med å sette et flagg: Hva hvis vi ønsker å stoppe en tråd som «sover», enten med `sleep()` eller `wait()` ?

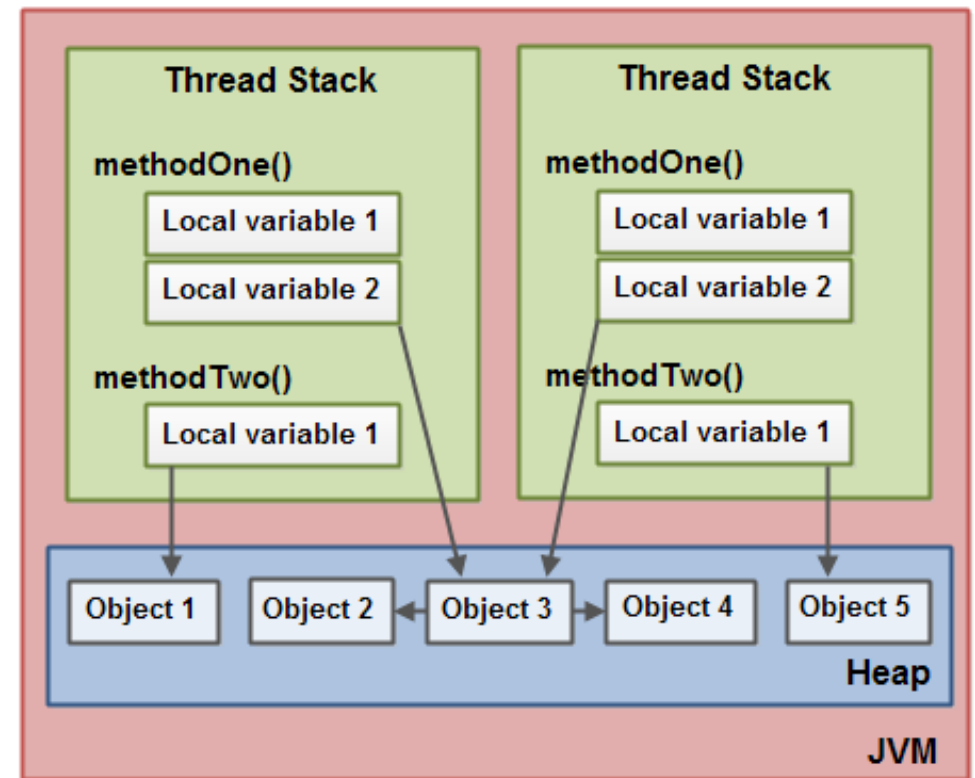
IntelliJ-prosjekt: 📁 f09-eksempler

Pakke for eksempelet: 📦 eks3_stoppeEnTraadSomSover



Java Memory Model – Delte vs. «lokale» data

- › For å oppnå **trådsikker** kode, så er første skritt å identifisere hvilke data som er **delte**, og hvilke som er **lokale** i den enkelte tråd.
- › Deretter må man sørge for **atomisitet**, **synlighet** og rett rekkefølge på de instruksjoner som skriver og leser de delte dataene.
- › Bruk av låsing med **synchronized** er ett verktøy vi har, og merking med **volatile** er et annet.
- › I dag skal også vi se på noen andre tilnærminger.



Nå: Ulike teknikker for å oppnå trådsikkerhet

- › Nå skal vi se på noen andre momenter som påvirker trådsikkerhet, og hvordan vi kan ta hensyn til disse nå vi programmerer.
- › Stikkord:
 - › Stateful eller Stateless
 - › Mutable eller Immutable
 - › Streams
 - › Atomic primitives
 - › Synchronized collections
 - › Concurrent collections

Lesestoff:

- › What is Thread-Safety and How to Achieve it? (av Alejandro Ugarte)
<https://www.baeldung.com/java-thread-safety>



Stateful eller Stateless

- › Ofte er feil i flertrådede applikasjoner resultat av feilaktig **deling av tilstand** mellom flere tråder (som resulterer i at tråder oppdaterer data feil).
- › Hva om vi lager programmer som har **mindre tilstand**, dvs. mer direkte beregninger? Eller **ingen tilstand**, kun beregninger?
- › Ser vi litt større på dette, kan vi si at programmering ved å endre tilstand på ting tilhører det **imperative** (f.eks. **objektorienterte**) paradigmet, mens det finnes andre måter å tenke på, f.eks. det **funksjonelle** paradigmet.
- › [Vise eksempel **ex1_stateless**]
- › Konkl: Stateless (tilstandsløs) implementasjon hjelper på trådsikkerhet.



Mutable eller Immutable

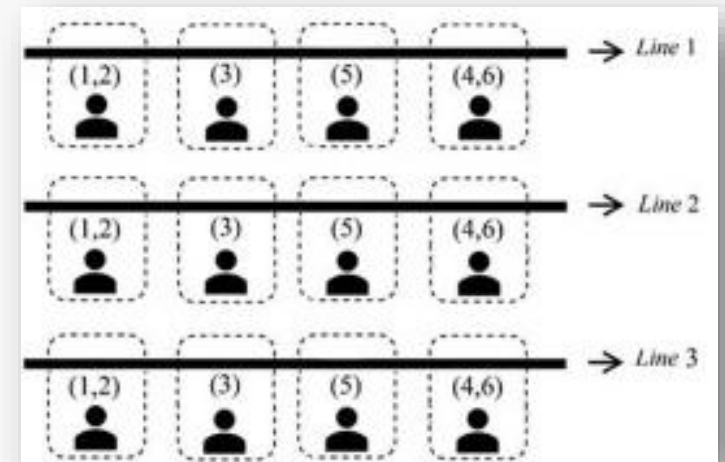
- › Hvis vi velger / trenger å dele tilstand mellom ulike tråder kan vi lage trådsikre klasser ved å lage dem **immutable** (uforanderlige).
- › Et objekt er immutable hvis objektets tilstand ikke kan endres etter at objektet er opprettet.
- › Et godt eksempel er String-objekter. Sjekk ut virkemåten til f.eks.:
 - › `concat(String s)`, `replace(String old, String new)`, `substring(int begin, int end)`,
 - › `toLowerCase()`, `toUpperCase()`, `trim()`, ...
- › [Vise eksempel **ex2_immutable**]
- › Konkl: Immutable (uforanderlig) implementasjon hjelper på trådsikkerhet.



Streams generelt

```
List<String> etternavnene = people.stream()           //FROM people
    .filter(p -> p.getAge() > 50)                   //WHERE age > 50
    .map(Person::getLastName)                         //SELECT lastName
    .toList();
```

- › people er en liste av Person-objekter
- › Å bruke people som en kilde til en stream vil IKKE mutere people-listen
- › , men vil produsere en NY liste som et resultat
- › Kunne vi satt opp flere samlebånd i parallell (i ulike tråder)?
- › Ja! Vi kan bruke `people.parallelStream(). ...`
<https://www.baeldung.com/java-when-to-use-parallel-stream>



Atomic objects

- › Dere husker da vi telte oppover og nedover samtidig, og vi ikke endte opp på 0 fordi **verdi++** ikke er en **atomisk operasjon**.
- › Det finnes noen klasser der slike operasjoner er atomiske. Disse er nyttige i flertrådede applikasjoner.
- › [Vise eksempel **ex3_atomic**]
- › Konkl: Atomic objects kan bruke som alternativ til synchronized.



Synchronized collections

- › Collections som ArrayList, HashSet, HashMap osv. er ikke trådsikre. Dvs. at hvis flere tråder deler en slik samling og prøver å oppdatere ting kan det gå galt.
- › Vi har jo lært om synchronized. Det kunne jo vært en løsning!? Men ytelse.
- › Hvis vi trenger en synkronisert samling har Java et API som "wrapper" disse samlingene i et skall av synkroniserte metoder, f.eks.:

```
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
```

- › [Vise eksempel

ex4_safe_collections]

- › Konkl: Trådsikre samlinger kan oppnås ved å bruke en wrapper.



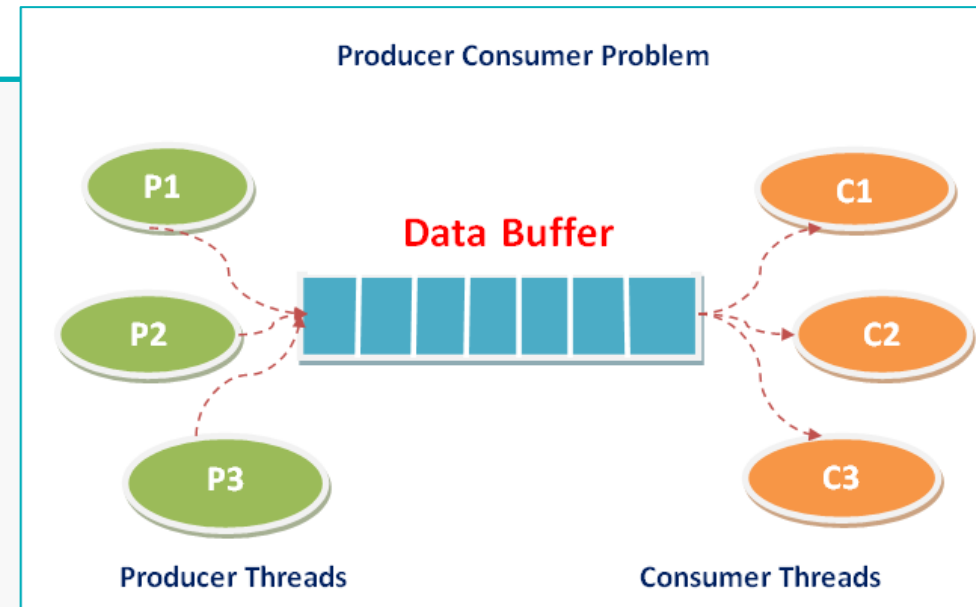
Concurrent collections

- › Et alternativ til synkroniserte samlinger er såkalte "concurrent collections".
- › Disse er laget med tanke på flertråding, har bedre ytelse enn de rene synkroniserte samlingene, og har også varianter med ekstra nytting funksjonalitet.
- › Eksempel: `Map<String,String> concurrentMap = new ConcurrentHashMap<>();`
- › Eller: `BlockingQueue<String> bq = new LinkedBlockingQueue<>(3);`
- › Skal se litt mer på BlockingQueue ...



Producer-Consumer og BlockingQueue

- › Dere husker denne fra sist (og fra Obligen)
- › Problemstillingen i Obligen er en variant av **Producer-Consumer**-problemstillingen.
- › Man har altså en buffer (en kø), og man har produsenter som leverer til køen, og konsumenter som tar ut av køen.
- › Dette er så pass vanlig at man har laget en egen datastruktur for dette.
- › En **BlockingQueue** er en slik (begrenset) buffer / kø, og den vil bruke **låsing** og **wait / notify** automatisk ved tom eller full kø.
- › [Vi får se på eks **ex5_conc_collections**]





Nå: Hva hvis vi vil at tråd skal returnere noe?

- › Det er en trend mot ikke-blokkerende, asynkron programmering
- › Har noe hørt om reactive programmering?
- › Thread/Runnable bruker en såkalt "shared state"-modell
- › Vi har sett på trådproblematikk hvis flere tråder deler data

- › Er det andre modeller ...? ... JA ...
- › Nå skal vi se litt videre enn det vi har sett på så langt
- › Lesestoff:
 - › Java Callable and Future Tutorial (av Rajeev Singh)
<https://www.callicoder.com/java-callable-and-future-tutorial/>



Eksempler på "å returnere noe" fra en tråd

- › Poenget med disse eksemplene er å se hvordan vi kan fortsette i main() mens vi venter på at en beregning gjør seg ferdig og returnerer et resultat.
- › Bruker naiv Fibonacci-beregning som eksempel på en beregning som tar tid (men burde kanskje funnet et annet/bedre eksempel).
- › Skal se på noen Java-klasser/intefaces: **ExecutorService**, **Callable**, **Future**, og **CompletableFuture**
 - › fut0_PlainUtenTraader
 - › fut1_RunnableOgThread
 - › fut2_RunnableOgExService
 - › fut3_CallableOgFuture
 - › fut4_BedreInnpakking
 - › fut5_FlereFibsSamtidig
 - › fut6_CompletableFuture

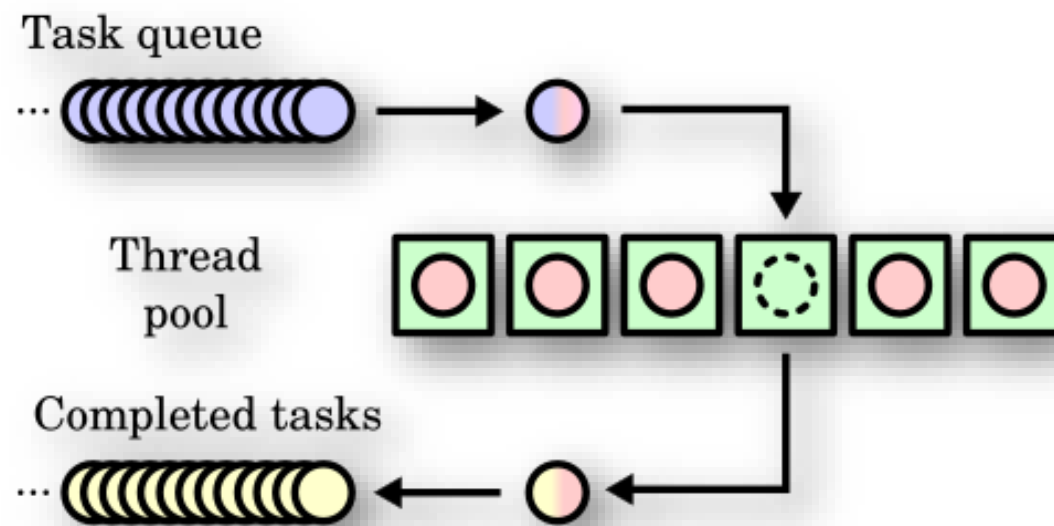


Resten av lysbildene er litt tekst som utfyller eksemplene med Callable, Futures ..

(Hentet fra Erlend sin forelesning, og må regnes som kursorisk)

Trådbassenger

Thread pools, eller trådbassenger på norsk, er en mekanisme i Java for å effektivt håndtere og gjenbruke tråder for å utføre oppgaver samtidig.



Trådbasseng: ExecutorService

1. Opprettelse av trådbasseng:
 - a. Kontrakt: `ExecutorService`
 - b. Implementasjon: `ThreadPoolExecutor` eller `Executors.newFixedThreadPool()`
2. Send inn oppgaver:
 - a. Gi `Executor.submit` en implementasjon av en `Runnable` eller `Callable`
3. Utførelse av oppgaver:
 - a. Trådbassenget håndterer utførelsen av oppgavene i bakgrunnen.
4. Gjenbruk av tråder:
 - a. En av fordelene med trådbassenger er gjenbruk av tråder.
5. Avslutning:
 - a. `Executor.shutdown()` (evt. `shutdownNow()`), men bør unngås.

Callable

Tråder som bruker Runnable kan ikke returnere noe...

Callable kan returnere noe!

```
executorService.submit(new Callable<>() {  
    @Override  
    public Integer call() {  
        return new Random().nextInt();  
    }  
});
```

Problem: Hvordan vet vi at retur verdien er klar? Løsning: Future

Future lar deg jobbe med oppgaver som kan ta tid, uten å blokkere hele programmet mens du venter på resultatet.

1. Innsending av Oppgave:

- Når du sender inn en oppgave til en tråd (thread) for utførelse, får du tilbake en Future.

2. Venting på Resultatet:

- Du kan bruke denne Future for å vente på resultatet av oppgaven, men du trenger ikke å stoppe og vente aktivt. Du kan fortsette å gjøre andre ting mens du venter.

3. Henting av Resultatet:

- Når oppgaven er ferdig, kan du bruke Future til å hente resultatet når du er klar.

CompletableFuture

Når du har oppgaver som tar tid å fullføre, vil du at applikasjonen din skal være responsiv og ikke blokkere hovedtråden mens du venter på resultatet. CompletableFuture lar deg starte oppgaver som kjører i bakgrunnen og hente resultatet når det er klart, uten å blokkere.

CompletableFuture

1. Opprette en CompletableFuture:
 - Du lager en CompletableFuture for å utføre en oppgave i fremtiden.
2. Utføre en Oppgave Asynkront:
 - Du kan starte en oppgave som utføres i bakgrunnen, og du trenger ikke vente på den.
3. Håndtere Resultatet:
 - Når oppgaven er ferdig, kan du hente resultatet eller utføre andre handlinger basert på resultatet.
4. Håndtere Feil:
 - Du kan også håndtere feil som kan oppstå under oppgavens utførelse.
5. Sammensette Oppgaver:
 - Du kan kombinere flere oppgaver eller kjøre dem i en sekvens.
6. Asynkron Programmering:
 - CompletableFuture er nyttig for å lage programmer som er raske og reaktive, spesielt når du jobber med oppgaver som kan ta tid.

Non-blocking (ikke-blokkerende) programmering:

- En måte å skrive kode på der tråder eller prosesser ikke venter på hverandre for å fortsette.
- I stedet kan flere oppgaver utføres samtidig uten å blokkere hverandre.
- Dette gjøre programm mer effektive og responsive.
- Non-blocking kode er vanlig i systemer som trenger å håndtere mange samtidige oppgave for å unngå at systemet "låser seg" når en oppgave tar tid å fullføre Eksempel.
 - Webservere
 - Brukergrensesnitt

Asynkron programmering

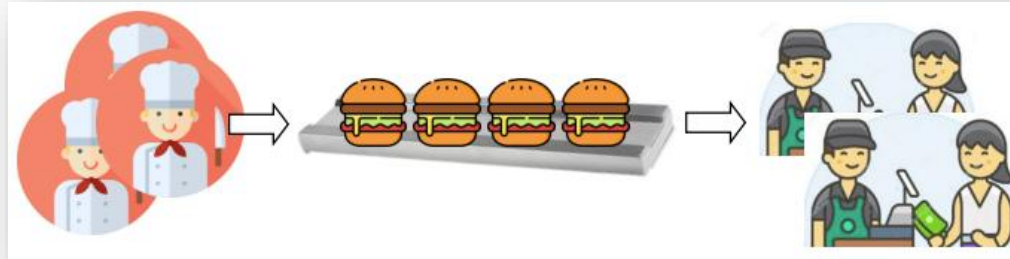
Asynkron programmering er en måte å håndtere oppgaver på uten å vente på at de skal fullføres før du går videre til neste. Dette gjør programmet raskere og mer responsivt ved å la det gjøre flere ting samtidig. Det brukes ofte i situasjoner der oppgaver tar tid, som å hente data fra nettet, uten å stoppe resten av programmet.

Reaktiv programmering

Reaktiv programmering er en programmeringsstil som fokuserer på å håndtere datastrømmer og hendelser på en smidig og responsiv måte. Den er spesielt egnet for å utvikle systemer som skal kunne håndtere store mengder data eller mange samtidige hendelser.

Videre

Jobbe med Oblig 2. **Innleveringsfrist søndag 20.09.**



Torsdag:

- › DAT108 Javahjelpen

Fredag:

- › Start modul **JavaScript** med lærer **Bjarte Kileng**

